

EZWorks

Reglas de Sesión y Autenticación

Documento de Arquitectura de Seguridad | Sprint 2

1. Propósito del Documento

Este documento define de manera precisa y ejecutable las reglas que gobiernan la autenticación de usuarios, la gestión del ciclo de vida de una sesión, y el comportamiento del sistema ante condiciones especiales como baneo, expiración de token, o cambios de estado en tiempo de ejecución.

No describe el modelo de datos ni los flujos de negocio; describe cómo el sistema verifica quién eres, qué puedes hacer, y qué ocurre cuando ese estatus cambia. Estas decisiones deben tomarse antes de iniciar Sprint 2, ya que son transversales a todos los módulos del backend.

2. Modelo de Autenticación

2.1 Estrategia de Token

EZWorks utilizará autenticación basada en JWT (JSON Web Token) con el siguiente esquema de dos tokens:

Token	Duración	Propósito	Almacenamiento en cliente
Access Token	15 minutos	Autorizar cada request a la API	Memoria del proceso (no localStorage)
Refresh Token	7 días	Obtener un nuevo Access Token sin re-login	Cookie HttpOnly + Secure (inaccesible desde JS)

▲ Razón	El Access Token debe vivir en memoria (no en localStorage ni sessionStorage) para evitar ataques XSS. El Refresh Token en cookie HttpOnly lo protege de ser robado por scripts maliciosos.
----------------	--

2.2 Payload del Access Token

El JWT debe contener únicamente los campos necesarios para la autorización. No incluir datos sensibles como contraseñas, números de documento, ni información financiera:

Campo	Tipo	Descripción
sub	UUID	ID único del usuario en la base de datos
role	string	Rol activo: 'EMPLEADOR', 'AYUDANTE' o 'ADMIN'
status	string	Estado de cuenta: 'ACTIVE', 'SUSPENDED', 'BANNED'
iat	timestamp	Fecha/hora de emisión del token (issued at)
exp	timestamp	Fecha/hora de expiración (15 min después de iat)

3. Ciclo de Vida de la Sesión

3.1 Inicio de Sesión

#	Paso
1	Cliente envía email + contraseña al endpoint POST /auth/login
2	Backend verifica hash de contraseña (bcrypt); si es incorrecto, retorna 401 con mensaje genérico 'Credenciales inválidas'
3	Backend verifica el status del usuario. Si está BANNED, retorna 403 con mensaje 'Tu cuenta ha sido inhabilitada' sin emitir token alguno
4	Si las credenciales son válidas y el status es ACTIVE o SUSPENDED, backend emite Access Token (body) y Refresh Token (cookie HttpOnly)
5	Cliente guarda el Access Token en memoria de la aplicación; nunca en localStorage

3.2 Renovación del Token (Refresh)

El Access Token expira cada 15 minutos. El cliente debe renovarlo de forma transparente sin forzar un re-login al usuario:

- El cliente implementa un interceptor HTTP que detecta respuestas 401 del servidor.
- Al recibir un 401, el interceptor llama automáticamente a POST /auth/refresh enviando el Refresh Token (que va automáticamente en la cookie).
- Si el Refresh Token es válido, el servidor emite un nuevo Access Token y lo retorna en el body de la respuesta.
- El interceptor reintenta el request original con el nuevo token de forma transparente para el usuario.
- Si el Refresh Token también está expirado o inválido, el servidor retorna 401. El interceptor redirige al login y limpia el estado de sesión.

Caso crítico	Si el usuario fue baneado mientras tenía sesión activa, el endpoint /auth/refresh también debe verificar el status actual del usuario desde la base de datos antes de emitir un nuevo token. No basta con verificar que el Refresh Token sea válido criptográficamente.
---------------------	---

3.3 Cierre de Sesión

- El cliente llama a POST /auth/logout.
- El servidor invalida el Refresh Token (marcándolo como revocado en una tabla refresh_tokens o usando una blacklist en Redis/cache).
- El servidor ordena al cliente borrar la cookie del Refresh Token con Set-Cookie de expiración en el pasado.
- El cliente limpia el Access Token de la memoria de la aplicación y redirige al login.

4. Verificación de Estado por Request

Todo endpoint protegido de la API debe ejecutar las siguientes verificaciones en el middleware de autenticación, en este orden exacto:

Orden	Verificación	Si falla: código HTTP	Mensaje al cliente
-------	--------------	-----------------------	--------------------

1	¿Existe el header Authorization: Bearer <token>?	401 Unauthorized	'Se requiere autenticación'
2	¿El token es criptográficamente válido (firma JWT correcta)?	401 Unauthorized	'Token inválido o malformado'
3	¿El token no ha expirado (campo exp)?	401 Unauthorized	'Sesión expirada'
4	¿El status del usuario en BD es ACTIVE o SUSPENDED (no BANNED)?	403 Forbidden	'Tu cuenta ha sido inhabilitada'
5	¿El rol en el token tiene permiso para este endpoint?	403 Forbidden	'No tienes permiso para esta acción'

Paso 4 — detalle crítico	La verificación del status del usuario (paso 4) debe hacerse contra la base de datos en cada request, NO solo al momento del login. Esto es lo que hace efectivo el baneo inmediato: el Access Token del usuario baneado seguirá siendo criptográficamente válido hasta que expire, pero el paso 4 lo rechazará de inmediato porque consulta el estado real en BD.
---------------------------------	--

5. Política de Sesiones Múltiples

Para el MVP, EZWorks adoptará la siguiente política de sesiones concurrentes:

- Se permite un máximo de 2 sesiones activas simultáneas por usuario (ej. celular + PC).
- Al iniciar una tercera sesión, el Refresh Token más antiguo se invalida automáticamente.
- El usuario cuya sesión fue invalidada verá el mensaje 'Tu sesión fue cerrada porque iniciaste sesión desde otro dispositivo' en el próximo request.

Alternativa simple	Si la implementación de múltiples sesiones es compleja para el Sprint 2, se puede adoptar política 'una sesión activa': al nuevo login se invalidan todos los Refresh Tokens anteriores del usuario. Documentar la política elegida en el código como comentario de arquitectura.
---------------------------	---

6. Restricciones de Acceso por Rol

6.1 Tabla de Permisos por Endpoint

La siguiente tabla muestra qué roles tienen acceso a las categorías principales de endpoints. El control de acceso debe implementarse en el backend; el frontend puede ocultarlo visualmente pero no es suficiente como control de seguridad.

Categoría de Endpoint	EMPLEADOR	AYUDANTE	ADMIN
Publicar / Editar vacante	✓ Sí	✗ No	✗ No
Postularse a vacante	✗ No	✓ Sí	✗ No
Seleccionar Ayudante (Match)	✓ Sí	✗ No	✗ No
Ver / Enviar mensajes de chat	✓ Solo chats propios	✓ Solo chats propios	✗ No (privacidad)
Ver perfil público de usuario	✓ Sí	✓ Sí	✓ Sí
Saldar deuda propia	✓ Sí	✓ Sí	✗ No

Panel de gestión disciplinaria	✗ No	✗ No	✓ Solo ADMIN
Crear / Eliminar categorías	✗ No	✗ No	✓ Solo ADMIN
Banear / Suspender usuarios	✗ No	✗ No	✓ Solo ADMIN

6.2 Regla de Propiedad de Recursos

Más allá del rol, el backend debe verificar la propiedad de cada recurso. Un Empleador autenticado no puede editar la vacante de otro Empleador, aunque ambos tengan el mismo rol. Para cada endpoint que modifique un recurso:

- Verificar que el sub del token (ID del usuario) sea el propietario del recurso solicitado.
- Si no es el propietario, retornar 403 Forbidden con mensaje 'No tienes permiso para modificar este recurso'.
- Nunca retornar 404 cuando el recurso existe pero el usuario no tiene acceso; usar siempre 403 para evitar revelar la existencia de recursos privados.

7. Restricción Funcional por Deuda (Inhabilitación)

La inhabilitación por deuda es un estado de cuenta diferente al baneo administrativo. Un usuario inhabilitado por deuda puede seguir iniciando sesión y navegando, pero no puede ejecutar las acciones principales de la plataforma.

Estado	Puede iniciar sesión	Restricciones
ACTIVE (deuda < límite)	✓ Sí	Sin restricciones adicionales
DEBT_LIMIT (deuda >= límite)	✓ Sí	No puede publicar vacantes, postularse, ni confirmar matches. Puede ver, saldar deuda y usar el chat de matches existentes.
SUSPENDED (admin)	✓ Sí (lectura)	Solo puede ver su cuenta. Todas las acciones bloqueadas. Puede leer chats activos.
BANNED (admin)	✗ No	No se emite token. Acceso total denegado.

La verificación del estado DEBT_LIMIT debe hacerse en los endpoints correspondientes (publicar vacante, postular, match), no en el middleware general de autenticación, ya que no afecta todos los endpoints.

8. Consideraciones de Seguridad para el MVP

- Contraseñas: Almacenar siempre con bcrypt (cost factor 12 mínimo). Nunca en texto plano ni con MD5/SHA1.
- Rate limiting: El endpoint POST /auth/login debe tener rate limiting (máximo 5 intentos fallidos por IP en 15 minutos) para prevenir fuerza bruta.
- HTTPS obligatorio: Toda comunicación entre cliente y servidor bajo HTTPS. El Refresh Token en cookie debe tener el atributo Secure para que nunca viaje por HTTP.
- Registro de IP en T&C: Según RF-19 (legal), registrar IP y timestamp al aceptar los Términos y Condiciones. Este dato NO debe incluirse en el JWT.
- CORS: Configurar CORS solo para el dominio de la aplicación; no usar wildcard (*) en producción.

9. Decisiones Pendientes (Requieren Respuesta del Equipo)

Las siguientes decisiones de arquitectura deben resolverse antes de iniciar Sprint 2. No pueden quedar sin definir porque impactan el esquema de base de datos y la implementación del middleware:

#	Pregunta	Opciones	Responsable
D-01	¿Cuántas sesiones simultáneas se permiten por usuario?	1 sesión / 2 sesiones / Sin límite	Ian (Scrum Master)
D-02	¿Qué ocurre con los matches activos cuando se banea a uno de los participantes?	Cancelar match / Dejar activo / Congelar	Yaider + William
D-03	¿Se puede cancelar un match confirmado? ¿Dentro de qué ventana de tiempo?	No cancelable / Cancelable con ventana / Cancelable siempre	William (PO)
D-04	¿El estado DEBT_LIMIT impide ver los perfiles de otros usuarios o solo las acciones transaccionales?	Solo acciones / Lectura reducida también	Yaider (Analista)